

StoryStackerTM Asset Pipeline & Folder Structure Guide

This document defines the recommended asset file structure, naming conventions, and workflow for the StoryStackerTM project.

The goal is to:

- Keep Blender source files editable and separate from game-ready exports.
- Centralize shared resources like textures, animations, and documentation.
- Support scalability to hundreds of shelves, objects, and books.
- Ensure compatibility with Unity or other game engines.

The structure follows a "conical" layout, with broad categories at the top level, narrowing to specific assets inside.

Conical Project Folder Layout

```
StoryStacker_Assets/  
|  
+-- 01_BlenderSource/           # Editable .blend files  
|   +-- Shelves/  
|   +-- Books/  
|   +-- Objects/  
|   +-- Environments/  
|   +-- Characters/  
|  
+-- 02_Textures/                # Textures & materials  
|   +-- Raw/                    # High-res source textures (PSD, TIF)  
|   +-- Optimized/              # Game-ready (PNG, WebP, JPG)  
|   +-- Materials/              # Material presets  
|  
+-- 03_Exports_GameReady/       # Files ready for engine import  
|   +-- FBX/                    # Includes empties/snap points  
|   +-- GLTF/                   # Alternate format  
|   +-- Prefabs/                # Unity prefabs  
|   +-- OBJ/                    # Static geometry backups  
|  
+-- 04_Animations/              # Animation source + exports  
|  
+-- 05_Audio/                   # Music, SFX, voice  
|  
+-- 06_Documents/               # Design docs, tech notes, diagrams  
|  
+-- 07_Temp/                    # Temporary or experimental files
```

Why Keep Assets in Centralized Folders?

1. Avoids duplication - shared textures or materials are stored once and reused.
2. Simplifies updates - change a texture in one location, all linked assets update.
3. Improves game performance - engines load shared resources faster.
4. Maintains clean workflow - clear division between source assets and game-ready files.
5. Supports scalability - easy to manage hundreds of assets with consistent naming.

Example: JukeBox Shelf Asset

```
01_BlenderSource/Shelves/Shelf_023_JukeBox.blend
02_Textures/Optimized/Shelf_023_JukeBox/jukebox_diffuse.png
03_Exports_GameReady/FBX/Shelf_023_JukeBox.fbx
04_Animations/Shelf_023_JukeBox/openDoor.fbx
06_Documents/TechDocs/SnapPoint_Map_Shelf_023_JukeBox.pdf
```

In this system, the JukeBox shelf's source model, textures, exports, animations, and documentation live in their respective top-level folders, linked by a consistent asset ID (Shelf_023_JukeBox).

Naming Convention

Assets use a consistent ID format for cross-folder linking:

[Category]_[ThreeDigitID]_[Name]

Examples:

- Shelf_023_JukeBox.blend
- snap_shelf023_slot01 (snap point empty in Blender)
- Book_107_MysteryTales.glb
- Obj_212_VintageLamp_diffuse.png

Three-digit numbering ensures correct sorting and easy reference in scripts.

Export Recommendations

- Export as FBX for Unity; ensure 'Include Empty Objects' is enabled to preserve snap/spawn points.
- Use GLTF/GLB for web builds or alternate engines.
- Keep OBJ exports for static meshes only; no animations or snap points.
- Always apply scale/rotation before export (Ctrl+A in Blender).